Bitmask DP

When n ≤ 20: let the state be "which items are already used"

Learning goals

- Recognize the **n ≤ 20** signal for exponential-in-n DP
- Encode subsets as integers; use bit operations fluently (Week 9)
- Solve **TSP / Hamiltonian paths** with `dp[mask][last]` in O(2^n · n²)
- Solve **assignment / matching** with `dp[mask]` in O(2^n · n)
- Enumerate **submasks** and prove the O(3^n) bound
- Use **(count, leftover)** pairs for packing problems (Elevator Rides)

Why bitmasks?

- Brute force over permutations: n! — 20! ≈ 2.4·10^18
- Many problems only care about **which** items are used, not **in what order**
- Subsets: 2^n — 2^20 ≈ 10^6 ✔

n	n!	2^n	2^n · n²	3^n
10	3.6·10^6	1024	10^5	5.9·10^4
16	2·10^13	65536	1.7·10^7	4.3·10^7
20	2.4·10^18	10^6	4·10^8	3.5·10^9

Bit toolkit (recap of Week 9)

```
mask >> i & 1            // is i in the set?
mask | (1 << i)          // add i
mask & ~(1 << i)         // remove i   (or mask ^ (1 << i) if present)
__builtin_popcount(m)    // set size
__builtin_ctz(m)         // index of lowest element
m & -m                   // lowest element as a bit
(1 << n) - 1             // full set
```

Mind precedence: write `(mask >> i) & 1`, and `1LL << i` for i ≥ 31.

Pattern 1: dp[mask][last] — TSP

Travelling salesman: n cities, distances `d[i][j]`; shortest tour visiting each city once and returning to 0.

```
dp[mask][v] = min cost of a path that starts at 0,
              visits exactly the cities in mask, and ends at v
```

- Base: `dp[{0}][0] = 0`
- Transition: `dp[mask ∪ {u}][u] ← dp[mask][v] + d[v][u]` for `u ∉ mask`
- Answer: `min over v of dp[full][v] + d[v][0]`

Why the order inside mask does not matter

Two paths with the same **set** of visited cities and the same **last** city have identical futures.

```
0 → 2 → 1 → 3      cost 40
0 → 1 → 2 → 3      cost 35   ← dominates; keep only this one
```

That's the whole trick: n! orders collapse into 2^n · n states.

TSP code (Held-Karp)

```
vector<vector<long long>> dp(1 << n, vector<long long>(n, INF));
dp[1][0] = 0;
for (int mask = 1; mask < (1 << n); mask++) {
    if (!(mask & 1)) continue;
    for (int v = 0; v < n; v++) {
        if (!(mask >> v & 1) || dp[mask][v] >= INF) continue;
        for (int u = 0; u < n; u++) {
            if (mask >> u & 1) continue;
            int nm = mask | (1 << u);
            dp[nm][u] = min(dp[nm][u], dp[mask][v] + d[v][u]);
        }
    }
}
```

Iterating masks in increasing order works: `nm > mask` always.

Complexity of dp[mask][last]

- States: 2^n · n
- Transitions per state: n
- **O(2^n · n²)** time, **O(2^n · n)** memory
- n = 20 → 4·10^8 simple ops (fine in C++ with tight loops) and 2·10^7 cells
- Memory: `int` 80 MB, `long long` 160 MB — check limits!

CSES 1690 — Hamiltonian Flights

Count routes 1 → n visiting every city exactly once (directed, multi-edges), mod 10^9+7. n ≤ 20.

- Same state, but **count** instead of min
- Pull form: `dp[mask][u] = Σ dp[mask \ {u}][v]` over flights `v → u`
- Pruning: skip masks without city 1; city n only in the full mask
- Store incoming adjacency lists → O(2^n · m)

Hamiltonian Flights code

```
dp[1][0] = 1;
for (int mask = 1; mask < (1 << n); mask++) {
    if (!(mask & 1)) continue;
    if ((mask >> (n-1) & 1) && mask != (1 << n) - 1) continue;
    for (int u = 1; u < n; u++) {
        if (!(mask >> u & 1)) continue;
        int prev = mask ^ (1 << u);
        long long ways = 0;
        for (int v : in[u])
            if (prev >> v & 1) ways += dp[prev][v];
        dp[mask][u] = ways % MOD;
    }
}
// answer: dp[(1 << n) - 1][n - 1]
```

Pattern 2: dp[mask] — assignment

n people, n jobs, `cost[i][j]`. Each person gets one job.

Key: person `i = popcount(mask)` is the **next** to be assigned — no need to store it!

```
dp[0] = 0;
for (int mask = 0; mask < (1 << n); mask++) {
    int i = __builtin_popcount(mask);        // next person
    if (i >= n) continue;
    for (int j = 0; j < n; j++)
        if (!(mask >> j & 1))
            dp[mask | 1 << j] = min(dp[mask | 1 << j], dp[mask] + cost[i][j]);
}
```

O(2^n · n). AtCoder DP O *Matching* counts perfect matchings this way.

Pattern 3: packing — CSES 1653 Elevator Rides

n ≤ 20 people, weights `w[i]`, capacity x. Minimum number of rides?

```
best[mask] = (rides, weight of last ride)   — lexicographically minimal
```

Add person p to the best arrangement of `mask \ {p}`:

- fits → same rides, `last + w[p]`
- does not fit → `rides + 1`, `last = w[p]`

Greedy "last ride as light as possible" is safe because the order of people inside a mask is free.

Elevator Rides code

```
vector<pair<int, long long>> best(1 << n, {INT_MAX, 0});
best[0] = {1, 0};
for (int mask = 1; mask < (1 << n); mask++)
    for (int p = 0; p < n; p++) {
        if (!(mask >> p & 1)) continue;
        auto [rides, last] = best[mask ^ (1 << p)];
        if (last + w[p] <= x) last += w[p];
        else { rides++; last = w[p]; }
        best[mask] = min(best[mask], make_pair(rides, last));
    }
// answer: best[full].first
```

O(2^n · n) instead of O(3^n).

Pattern 4: submask enumeration

Iterate over all `sub ⊆ mask`, from largest to smallest:

```
for (int sub = mask; sub > 0; sub = (sub - 1) & mask) {
    // use sub (non-empty)
}
```

`(sub - 1) & mask`: subtract 1 (borrow through low bits), then keep only bits of mask → next smaller submask.

Why O(3^n) total?

Summing over all masks:

```
Σ_mask 2^|mask| = Σ_k C(n,k) 2^k = (1 + 2)^n = 3^n
```

Each element is in one of 3 states: not in mask / in mask but not in sub / in sub.

n = 16 → 4.3·10^7 ✔, n = 20 → 3.5·10^9 ✘ (usually)

Partition DP — AtCoder DP U *Grouping*

Split n ≤ 16 rabbits into groups; score = Σ a[i][j] over pairs in the same group. Maximize.

```
// score[mask] precomputed in O(2^n · n)
dp[0] = 0;
for (int mask = 1; mask < N; mask++) {
    int low = mask & -mask;
    for (int sub = mask; sub > 0; sub = (sub - 1) & mask)
        if (sub & low)       // group of the lowest element: avoids repeats
            dp[mask] = max(dp[mask], dp[mask ^ sub] + score[sub]);
}
```

O(3^n).

Precomputing subset values in O(2^n · n)

```
for (int mask = 1; mask < N; mask++) {
    int low = __builtin_ctz(mask), rest = mask ^ (1 << low);
    score[mask] = score[rest];
    for (int j = 0; j < n; j++)
        if (rest >> j & 1) score[mask] += a[low][j];
}
```

Reuse the value of the mask without its lowest element. Same idea gives subset sums in O(2^n).

Recognizing bitmask DP

- n ≤ 20 (or ≤ 16 for 3^n, ≤ 24 with care) — strongest signal
- "visit every", "use each exactly once", "assign", "partition into groups"
- Small side of a grid (≤ 10 columns): profile DP, CSES 2181 *Counting Tilings*
- Meet in the middle when n ≈ 40 (Week 3)

Common pitfalls

- `1 << n` with n ≥ 31 → use `1LL`
- Operator precedence: `mask & 1 << i` is `mask & (1 << i)` ✔, but `mask >> i & 1 == 0` ✘
- Memory: `dp[1<<20][20]` of `long long` = 160 MB
- Counting the same partition many times → fix the lowest element's group
- Iterating masks in wrong order with "push" transitions (must go to larger masks)
- Forgetting the empty submask (or including it when it means "no progress" → infinite loop)

Practice problems

- CSES 1690 *Hamiltonian Flights*
- CSES 1653 *Elevator Rides*
- CSES 2181 *Counting Tilings* (profile DP)
- AtCoder DP Contest O *Matching*, U *Grouping*
- Codeforces 11D *A Simple Task* (count cycles)
- Codeforces 580D *Kefa and Dishes*
- Codeforces 16E *Fish* (probability over masks)
- Codeforces 8C *Looking for Order*
- Codeforces 1238E *Keyboard Purchase*

Summary

- n ≤ 20 → state = subset as an integer
- `dp[mask][last]` for paths (TSP, Hamiltonian): O(2^n n²)
- `dp[mask]` with implicit index = popcount (assignment): O(2^n n)
- Pair states (rides, leftover) for packing
- Submask enumeration: `sub = (sub - 1) & mask`, total O(3^n)
- Next week: interval DP & digit DP